



CÔNG NGHỆ LẬP TRÌNH WEB

- Số tín chỉ học phần: 3 TC (2LT, 1TH)
- Số tiết học phần: 60 tiết
- Học phần tiên quyết/học trước:
 - ✓ Lập trình hướng đối tượng (Java core+Java Swing)
 - ✓ C# (core, Windows form)
 - ✓ Cơ sở lập trình Web (HTML/CSS/JS. JQuery)

- Công nghệ lập trình Web là học phần chuyên ngành bắt buộc về lập trình nền tảng Web phía back-end với sinh viên ngành CNTT.
- Học phần này trang bị cho sinh viên các kiến thức về lập trình xử lý nghiệp vụ phía máy chủ, tương tác với cơ sở dữ liệu, quản lý cơ sở dữ liệu.

- Sinh viên sẽ được trang bị kiến thức và kỹ năng về Web back-end, hiểu và vận dụng được các công nghệ mới như: Web API, Web MVC, Restful API, Entity để tương tác với back-end, cơ sở dữ liệu và front-end, có khả năng triển khai Web back-end lên hosting.
- Sinh viên hiểu được các yêu cầu thiết kế Clean architecture, kiến trúc dịch vụ Microservice, triển khai code theo design pattern và quy trình DevOp (Docker, Serverless).
- Sinh viên hiểu được yêu cầu xây dựng một Web an toàn.

- Môn Công nghệ lập trình Web tập trung vào Web back-end, sử dụng C# triển khai trên framework .NET core hoặc tương đương (ngôn ngữ lập trình framework back-end)
- Sinh viên đã được học các nội dung 1+2 và cần ôn tập lại ở các môn tương ứng (OOP (Java core, Java Swing), C#, Cơ sở lập trình Web) và học mới nội dung 3 ở môn Công nghệ lập trình Web.

1. Sử dụng C# core, Java core

- + Entity framework
- + Design Pattern
- + Kiến trúc MVC
- + Kiến trúc xử lý thông điệp, dịch vụ
- + Async/await, bất đồng bộ
- + Functional programming
- + Biểu thức chính quy trong tìm kiếm văn bản
- + Lập Unit test

2. Cơ sở lập trình Web

- + fetch: gọi Web API từ front-end
- + Async/await: trong JS
- + HTML form với thuộc tính action, method POST/GET...
- + Truyền tham số qua URL
- + Lỗ hổng bảo mật Web, Web an toàn

3. Nội dung mới

- + Web API .Net core
- + MVC .Net core
- + RESTFull API
- + Kiến trúc Microservice
- + Clean architecture
- + Web security
- + Severless

- Đánh giá chuyên cần: 5%
- Đánh giá quá trình: 45%
- Đánh giá cuối kỳ: 50% (e-Project 1-5 sinh viên)

Đánh giá quá trình: 45%

- Ba bài trắc nghiệm: 20%
- Bài tập cá nhân hoặc nhóm: 5%
- Thực hành trên máy: 20%

Đánh giá quá trình: 45%

Các bài kiểm tra gồm:

- Kiến thức về lập trình Web back-end
- Kiến thức về lập trình Web API và xây dựng Web API bằng các nền tảng như .Net Core hoặc framework tương đương, ...
- Lập trình tương tác với cơ sở dữ liệu và trao đổi dữ liệu với client
- Kiến thức tổng hợp về lập trình triển khai Website lên môi trường internet

TECH STACK PREFERENCE

VIETNAM
IT & TECH TALENT LANDSCAPE
2024 - 2025 Vietnam IT Market Report

Javascript

1. Bun	64.30%
2. NodeJS	48.20%
3. Angular	45.10%
4. ReactJS	36.0%
5. Typescript	32.9%
6. Vue	31.4%

Java

1. Spring Boot	45.20%
2. Hibernate	7.00%
3. Struts	1.20%
4. Vaadin	1.20%

Python

1. Django	35.10%
2. Pandas	31.00%
3. Flask	19.10%
4. PyTorch	16.90%
5. TensorFlows	11.40%

.NET/ C#

1. .Net core	46.20%
2. ASP.Net core	37.00%
3. .Net framework	29.00%
4. ASP.Net MVC	16.50%
5. Xamarin	0.70%

PHP

1. Laravel	60.50%
2. Symfony	24.70%
3. CodeIgniter	17.10%
4. Yii	4.50%
5. CakePHP	4.00%

DevOps

1. Linux	76.50%
2. Docker	52.70%
3. Bash	11.90%
4. Kubernetes	10.30%

Mobile Development

1. Java	50.20%
2. Swift	30.50%
3. Flutter	19.10%
4. React Native	10.20%
5. Kotlin	3.80%

Database technologies

1. MySQL	78.00%
2. SQL Server	59.90%
3. MongoDB	36.10%
4. PostgreSQL	31.40%
5. Redis	21.90%

Cloud Platform

1. AWS	38.30%
2. Microsoft Azure	25.20%
3. VMWare	17.10%
4. Firebase	14.60%
5. Google Cloud Platform	8.60%

- **JavaScript** is the most popular language together with other IT skills. New "**Bun**" introduced in 2023 has quickly become a phenomenon attracting many users, stealing the crown of the previous Top 3 - Nodejs, Angular & ReactJS.
- Top choice to work with **Java** is Spring Boot/ Spring then Hibernate & Struts respectively.
- There hasn't been much change in **.NET/C#** list in which .NET Framework, .NET Core & ASP.NET Core top the list.
- Laravel remains the most popular framework in the **PHP** tech stack with more than 75%, followed by CodeIgniter & Symfony.
- Django, Pandas & Flask are top 3 choices related to **Python** stack
- MySQL keeps leading the SQL tech stack with more than 78%. Regarding mobile development, **Java/ Swift & Flutter** are main choices.

FRONT-END DEVELOPER

Foundations:

- HTML, CSS, JavaScript
- Responsive Design
- Version Control (Git)

Core Skills:

- Frameworks (React, Angular, Vue.js)
- State Management (Redux, Vuex)
- Build Tools (Webpack, Babel)

Advanced:

- Testing (Jest, Cypress)
- Performance Optimization
- Progressive Web Apps (PWA)

MOBILE DEVELOPER

Foundations:

- Core Programming Languages (Swift, Kotlin, Java)
- UI/UX Design Principles
- Version Control (Git)

Core Skills:

- Cross-Platform Tools (React Native, Flutter)
- API Integration
- Performance Tuning

Advanced:

- AR/VR Development
- Advanced Networking (WebSockets, Bluetooth)
- Security & Encryption

BACKEND DEVELOPER

Foundations:

- Programming Languages (Node.js, Java, Python, Ruby, PHP, C#/.Net, Go etc.)
- Database Management (SQL, NoSQL)
- RESTful APIs, GraphQL

Core Skills:

- Authentication & Authorization
- ORM (Object-Relational Mapping)
- Server Management (Linux, Nginx)

Advanced:

- Microservices Architecture
- Caching Strategies (Redis, Memcached)
- CI/CD Pipelines

CYBERSECURITY ANALYST

Foundations:

- Networking (TCP/IP, VPN)
- Operating Systems Security
- Scripting (Python, PowerShell)

Core Skills:

- Threat Analysis
- Penetration Testing (Metasploit, Burp Suite)
- SIEM Tools (Splunk, ELK)

Advanced:

- Incident Response
- Forensics & Malware Analysis
- Security Architecture

AI/ML ENGINEER

Foundations:

- Programming (Python, R)
- Mathematics & Algorithms
- Data Wrangling

Core Skills:

- Machine Learning Models (Supervised, Unsupervised)
- Neural Networks & Deep Learning
- NLP (Natural Language Processing)

Advanced:

- Reinforcement Learning
- AI Ethics & Bias
- Model Deployment & Monitoring

DATA SCIENTIST

Foundations:

- Mathematics & Statistics
- Programming (Python, R)
- Data Wrangling (Pandas, NumPy)

Core Skills:

- Data Visualization (Matplotlib, Seaborn)
- Machine Learning (Scikit-learn, TensorFlow)
- SQL & NoSQL Databases

Advanced:

- Deep Learning (Keras, PyTorch)
- Big Data Tools (Hadoop, Spark)
- Model Deployment

CLOUD ENGINEER

Foundations:

- Networking Basics
- Scripting (Python, Bash)
- Virtualization & Containers (Docker)

Core Skills:

- Cloud Services (AWS, Azure, GCP)
- Infrastructure as Code (Terraform, CloudFormation)
- Security & Compliance

Advanced:

- Serverless Architecture (Lambda, Azure Functions)
- Multi-cloud Management
- Cost Optimization

DEVOPS ENGINEER

Foundations:

- Operating Systems (Linux, Windows)
- Networking Concepts
- Scripting (Bash, Python)

Core Skills:

- CI/CD Tools (Jenkins, GitLab CI)
- Containerization (Docker, Kubernetes)
- Cloud Platforms (AWS, Azure, GCP)

Advanced:

- Infrastructure as Code (Terraform, Ansible)
- Monitoring & Logging (Prometheus, ELK Stack)
- Security Best Practices

NETWORK ENGINEER

Foundations:

- Networking Basics (TCP/IP, DNS)
- Routing & Switching (Cisco, Juniper)
- Network Security

Core Skills:

- Wireless Networks
- Network Monitoring Tools
- VPNs & Firewalls

Advanced:

- Software-Defined Networking (SDN)
- Cloud Networking
- High Availability (HA) & Disaster Recovery

1. Cơ sở dữ liệu và Quản lý

- ✓ SQL Server 2019 hoặc mới hơn
- ✓ SQL Server Management Studio (SSMS) 2022

2. Môi trường Phát triển

- ✓ Visual Studio 2022
- ✓ .NET Core/.NET: Tối thiểu: .NET 6.0
- ✓ Khuyến nghị: .NET 8.0 (phiên bản mới nhất và ổn định)

3. Trình duyệt Web hỗ trợ

- ✓ Microsoft Edge, Google Chrome, Mozilla Firefox

- **Với Web API .NET Core:**

- ✓ HTML
- ✓ CSS
- ✓ JavaScript
- ✓ jQuery

- **Với MVC .NET Core:**

- ✓ Razor Pages (*.cshtml)
- ✓ Các view engine tích hợp sẵn của .NET Core

Khuyến nghị tự học thêm (không bắt buộc): AngularJS, ReactJS, VueJS, NodeJS, Flutter,...

- + **Posmant**: kiểm tra Web API
- + **Docker**: đóng Web apps để triển khai
- + **Figma**: vẽ UI cho Web fron-tend
- + **VueJS, AngularJS, hoặc ReactJS**: thiết kế Web front-end
- + **Visual Studio .NET**: Lập trình Web back-end .Net core
- + **SQLServer, MySQL** với SQLServer Authentication
- + Đăng ký tài khoản Github, sử dụng Github Actions, thiết lập CI/CD pipelines

- Có máy tính
- Cài đặt đầy đủ môi trường
- Có kỹ năng lập trình Web front-end
- Có kỹ năng về sử dụng cơ sở dữ liệu

- [1] Tamir Dresher, Amir Zuker, Shay Friedman, Hands-On Full-Stack Web Development with ASP.NET Core, ISBN:9781788627757, 178862775X, Publisher:Packt Publishing, Published:October 31, 2018.
- [2] Ravi Kant Soni, Full Stack AngularJS for Java Developers, ISBN:9781484231982, 1484231988, Publisher:Apress, Published:December 4, 2017
- [3] Mark Heckler, Spring Boot: Up and Running, ISBN:9781492076957, 1492076953, Publisher:O'Reilly Media, Published:February 5, 2021

Phạm vi và tiêu chí đánh giá kết quả học tập của môn học

Bài tập cá nhân hoặc nhóm (đề tài assignment)

Trình bày và lập chương trình thử nghiệm về một trong các công nghệ sau trên Web API và Web MVC:

- Web front-end AngularJS/VueJS/ ReactJS/NodeJS và Web API .NET core hoặc framework tương đương.
- Web MVC với Blazor hoặc framework tương đương.
- Unit test.
- CI/CD.

- Design Pattern.
- RabbitMQ/Kafka.
- Clean Architecture.
- Microservices.
- Docker.
- Serverless.
- Web Security

- Yêu cầu:
 - ✓ Tối đa 2 sinh viên/nhóm
 - ✓ Thời gian nộp lần đầu để review: tiết thứ 45
- Sản phẩm nộp:
 - ✓ Tài liệu báo cáo file .docx
 - ✓ Clip quay code chạy (nếu có)
 - ✓ File .pptx trình bày
 - ✓ Code chạy, Databases... (nếu có)

(Khuyến khích dùng AI hỗ trợ khi làm .docx, .pptx và clip)

- Yêu cầu:
 - Tối đa 4 sinh viên/nhóm
 - Thời gian nộp lần đầu để review: tiết thứ 50
- Sản phẩm nộp:
 - ✓ Tài liệu file .docx: Phân tích thiết kế, triển khai, hướng dẫn sử dụng.
 - ✓ Clip quay code chạy
 - ✓ File .pptx trình bày
 - ✓ Code chạy, Databases...

(Khuyến khích dùng AI hỗ trợ khi làm.docx, .pptx và clip)

Tiêu chí cộng điểm

- **Nhóm 1.** AngularJS/Vuejs/ReactJS/NodeJS/Flutter..., UI thiết kế phù hợp với Figma
- **Nhóm 2.** Code Web: Design pattern (mẫu DAO, và ít nhất 3 mẫu khác), SOLID, clean code, clean struter.
- **Nhóm 3.** Thiết kế kiến trúc: Clean structer, Microservices, tích hợp công nghệ phức tạp (RabbitMQ/Kafka, Kubernetes, giải pháp tối ưu hiệu năng, giải pháp Scalable Web Application...)
- **Nhóm 4.** Tích hợp AI, dữ liệu thật, nghiệp vụ thực tế (có địa chỉ áp dụng thật sự).

Tiêu chí cộng điểm

- **Nhóm 5.** Đảm bảo tính an toàn, có giải pháp phòng chống các lỗ hổng bảo mật Web
- **Nhóm 6.** DevOp (host thật, build docker, triển khai serverless...)
- **Nhóm 7.** Có Unit test, CI/CD...
- **Nhóm 8.** Có tài liệu User story, đặc tả yêu cầu phần mềm (SRS), tài liệu kiểm thử theo V-model...

Điểm đánh giá project

- Thang điểm: 10
- Nếu project không thực hiện các nội dung trong các nhóm cộng điểm, điểm tối đa 7

Khuyến khích dùng AI để làm.docx, .ppt và clip:

K1. Sử dụng Figma, Canva hỗ trợ thiết kế UI và trình bày tài liệu.

K2. Sử dụng AI để tạo ảnh, video cho nội dung Web

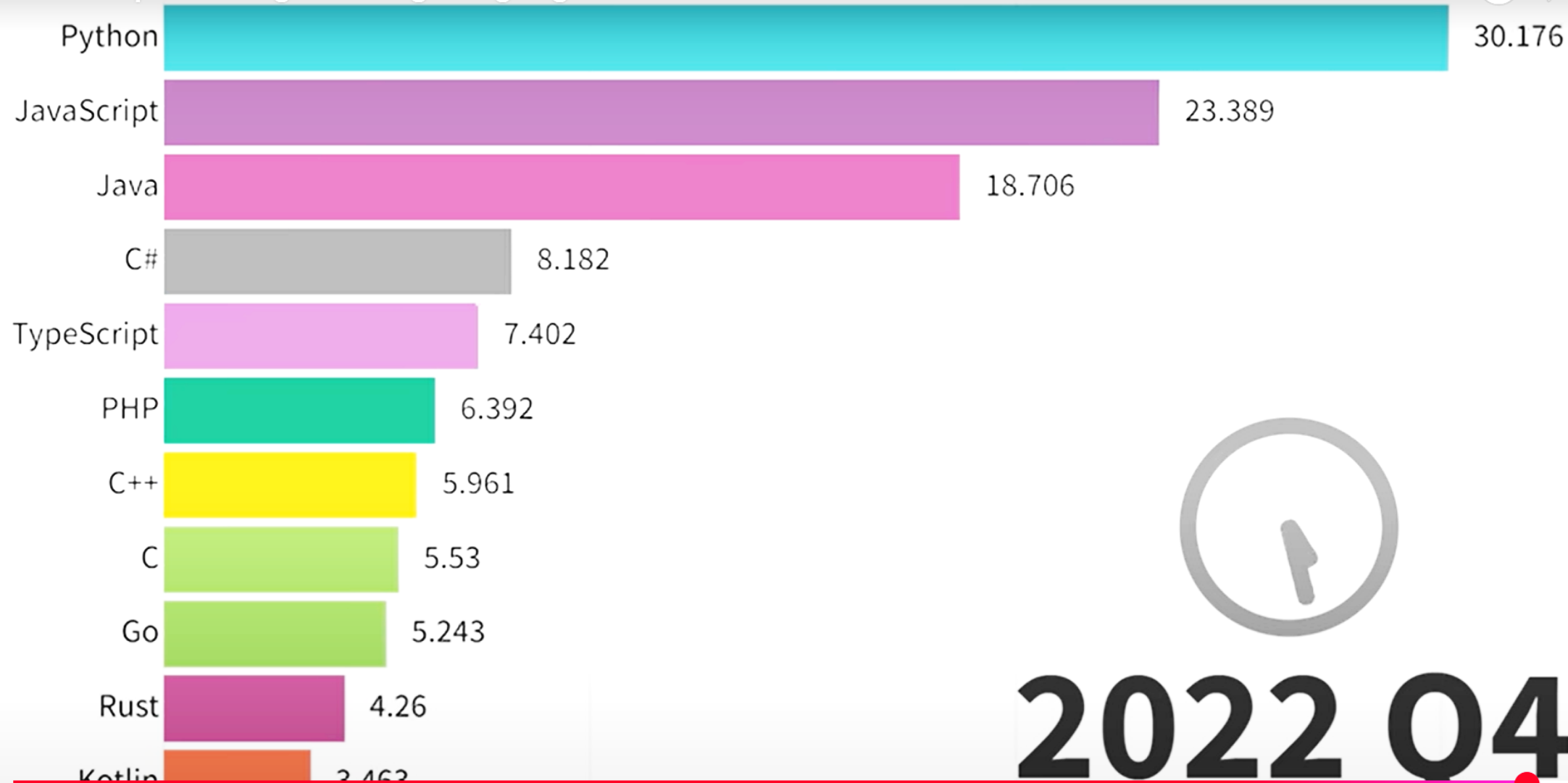
K3. Sử dụng AI để sinh code (chatGPT, Copilot).

K4. Sử dụng AI viết khung tài liệu

K5. Sử dụng AI mô phỏng data thử nghiệm và mô phỏng kịch bản sử dụng

Sự phát triển các ngôn ngữ giai đoạn 1965-2022

Most Popular Programming Languages 1965 - 2022



Web back-end và **Web fullstack**
là 1 môn học khác hoàn toàn
môn học khác, học tốt có luôn
tương ứng vị trí việc làm ở
doanh nghiệp phần mềm

Ước tính **300 giờ để có được những kiến thức cơ bản về back-end .NET.**

- * Học C#: Ngôn ngữ lập trình chính để phát triển ứng dụng .NET.
- * Tìm hiểu về .NET Framework hoặc .NET Core: Nền tảng để xây dựng các ứng dụng.
- * Làm quen với các công cụ phát triển: Visual Studio, .NET CLI.
- * Học về cơ sở dữ liệu: SQL Server, Entity Framework.
- * Tìm hiểu về các kiến trúc ứng dụng: MVC, API, Microservices.
- * Thực hành: Xây dựng các dự án nhỏ để củng cố kiến thức.

- * Lập kế hoạch học tập rõ ràng: Chia nhỏ mục tiêu thành các phần nhỏ hơn và theo dõi tiến độ.
- * Thực hành thường xuyên: Viết code càng nhiều càng tốt.
- * Tham gia cộng đồng: Đặt câu hỏi, chia sẻ kiến thức với những người khác.
- * Tìm một mentor: Một người có kinh nghiệm sẽ giúp định hướng và giải đáp thắc mắc.

Một số nguồn tài liệu tham khảo:

- * Microsoft Docs: Tài liệu chính thức của Microsoft về .NET.
- * Pluralsight, Udemy, Coursera: Các nền tảng cung cấp nhiều khóa học về .NET.
- * YouTube: Có rất nhiều kênh dạy lập trình .NET miễn phí.
- * Các diễn đàn, group: Stack Overflow, .NET Việt Nam.



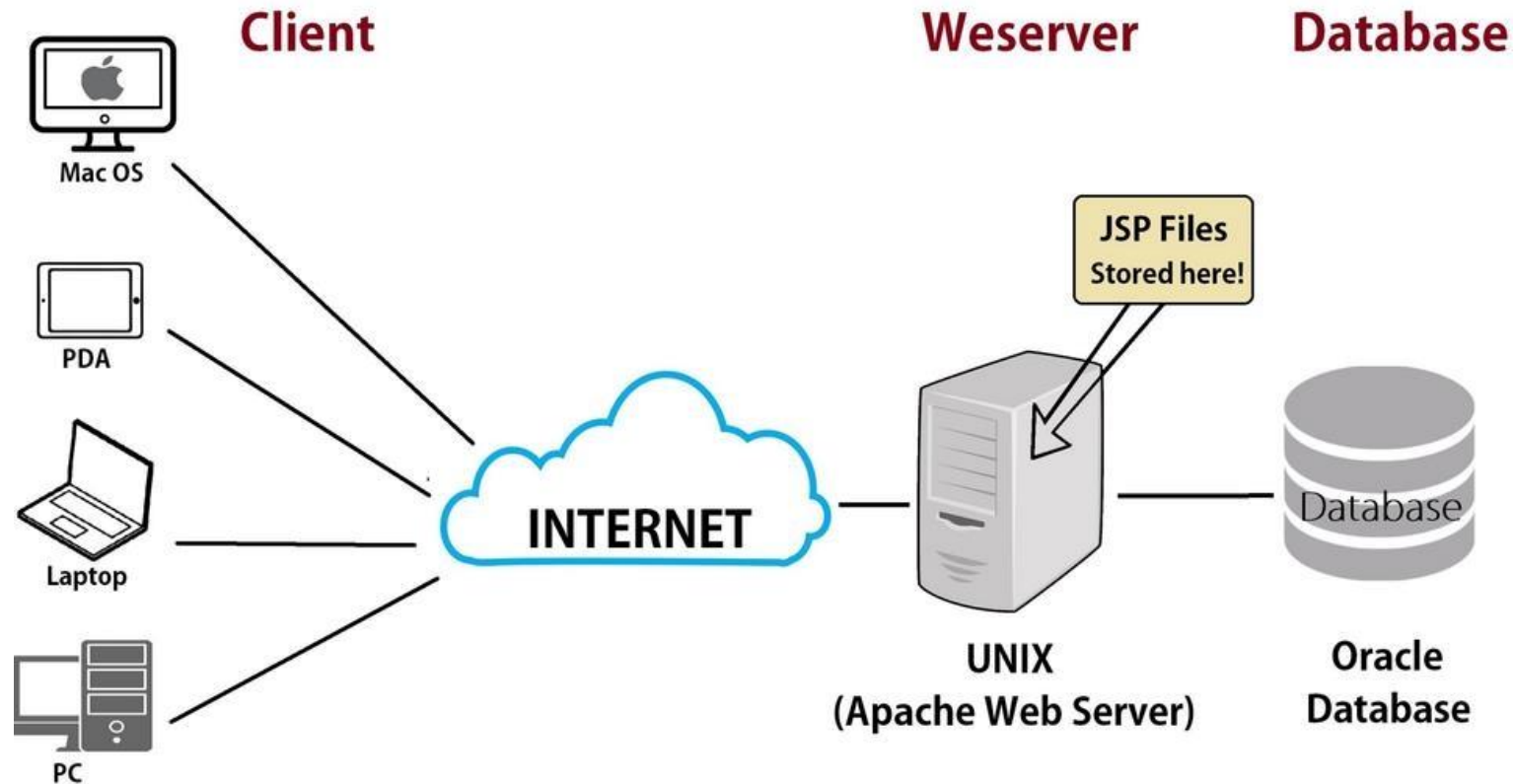
Chương 1. Tổng quan về lập trình Web back-end

- 1.1. Giới thiệu về lập trình Web
- 1.2. So sánh lập trình Web với các loại lập trình khác
- 1.3. Sự khác biệt giữa front-end và back-end
- 1.4. Vị trí việc làm (Web Developer Full-Stack, Back-end Developer, DevOps Engineer, API Developer)
- 1.5. Ôn tập kiến thức
- 1.6. Phạm vi và tiêu chí đánh giá kết quả học tập của môn học
- 1.7. Giới thiệu các framework trong môn học (Back-end, Front-end, ORM, DevOp)

- Giới thiệu về lập trình Web
 - Khái niệm lập trình Web và lập trình Web back-end
 - Mô hình client-server
 - Tầm quan trọng của lập trình Web và lập trình Web back-end
- So sánh lập trình Web với các loại lập trình khác
 - So sánh với lập trình ứng dụng desktop (Winform)
 - So sánh với lập trình app mobile

- Sự khác biệt giữa front-end và back-end
 - Front-end và các ngôn ngữ lập trình front-end
 - Back-end và các ngôn ngữ lập trình back-end
 - Sự khác biệt giữa back-end và front-end
- Vai trò của Full stack developer
 - Lập trình full-stack
 - Sự khác biệt với lập trình từng thành phần của ứng dụng hoặc Website
 - Giới thiệu về kiến trúc MVC

Mô hình Client Server



Mô hình mạng Client Server là mạng máy tính có cấu trúc gồm các thiết bị hoặc chương trình giao tiếp thông tin. Mạng lưới gồm hai thành phần chính là Client (máy khách) và Server (máy chủ)

- **Máy khách (Client)**

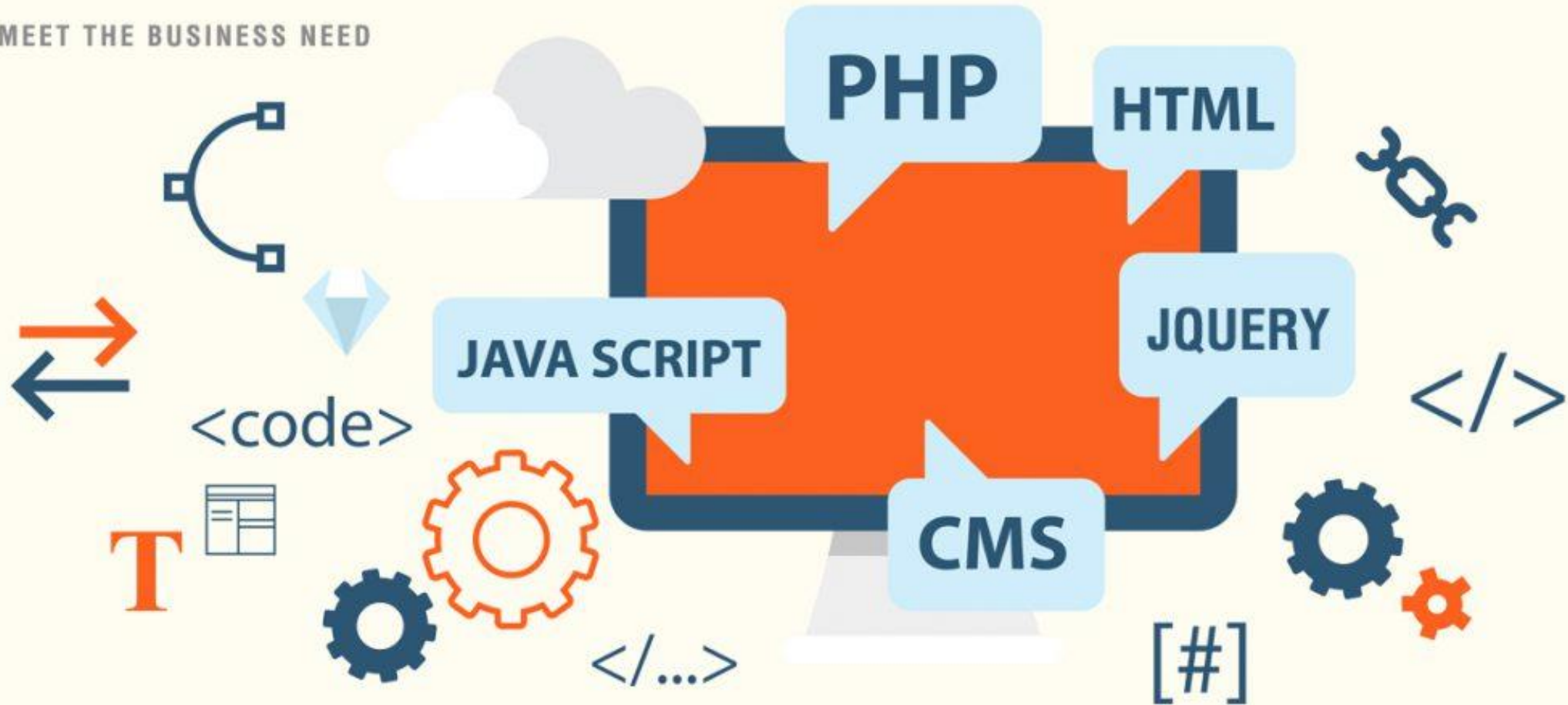
- ✓ Là thiết bị hoặc chương trình yêu cầu dịch vụ từ máy chủ.
- ✓ Gửi yêu cầu đến máy chủ và nhận phản hồi từ máy chủ sau khi yêu cầu được xử lý.

- **Máy chủ (Server)**

- ✓ Là thiết bị hoặc chương trình đảm bảo trách nhiệm cung cấp dịch vụ cho khách hàng.
- ✓ Có thể thực hiện nhiệm vụ lưu trữ dữ liệu, xử lý yêu cầu từ máy khách, cung cấp dịch vụ như email, trang Web, trò chơi trực tuyến và nhiều ứng dụng khác.
- ✓ Máy chủ luôn hoạt động và phản hồi khi có yêu cầu từ máy khách.

WEB DEVELOPMENT

MEET THE BUSINESS NEED

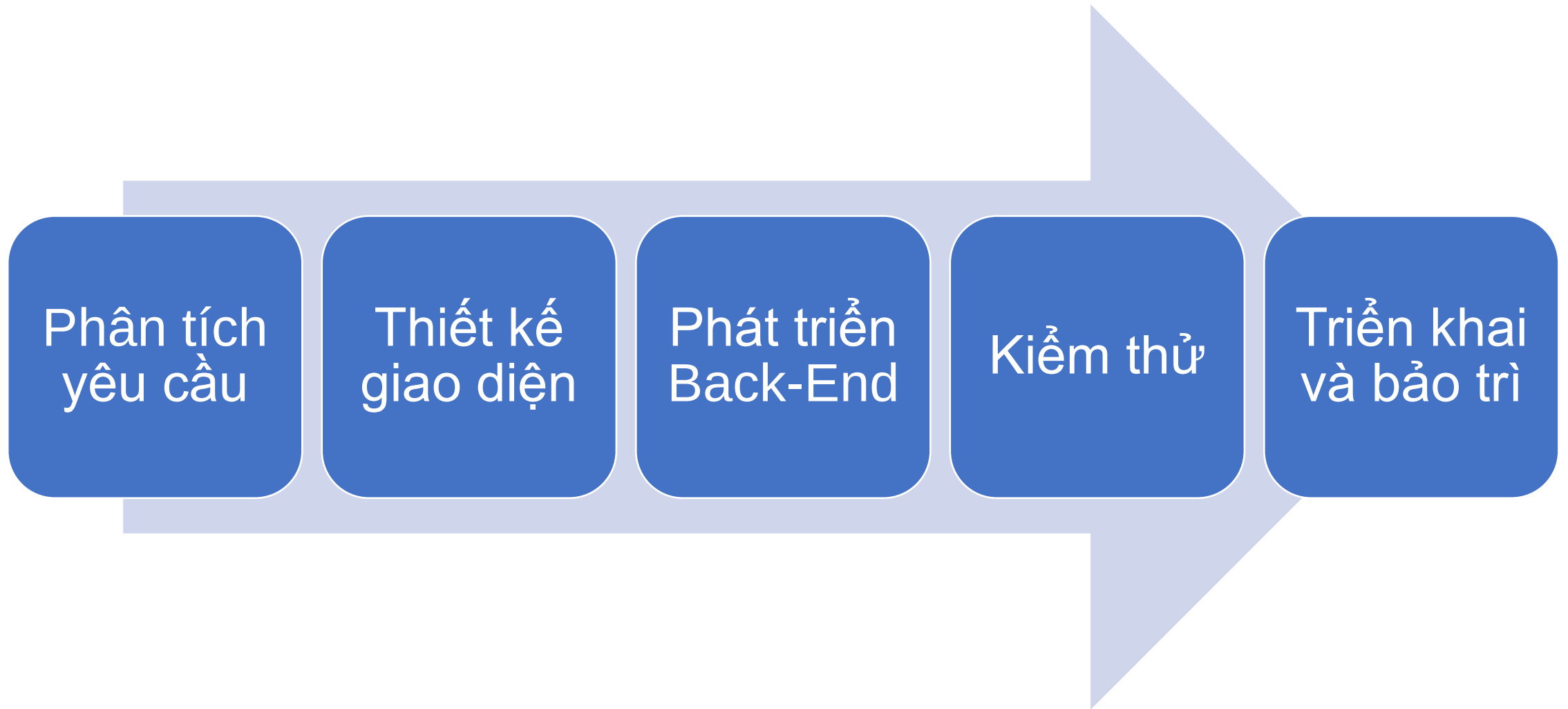


Lập trình Web là quá trình xây dựng và phát triển các ứng dụng hoặc trang Web chạy trên trình duyệt, sử dụng các công nghệ lập trình khác nhau. Đây là một nhánh của lập trình ứng dụng, tập trung vào việc thiết kế giao diện (front-end), xử lý logic (back-end), và quản lý dữ liệu để đáp ứng nhu cầu tương tác của người dùng trên internet.

1. Front-End (Giao diện người dùng)
2. Back-End (Phần xử lý server-side)
3. Cơ sở dữ liệu (Database)

- 1. Lập trình Web tĩnh (Static Web Development)**
- 2. Lập trình Web động (Dynamic Web Development)**
- 3. Lập trình Full-Stack Web**

Quy trình lập trình Web



1. Ngôn ngữ lập trình phổ biến

- HTML, CSS, JavaScript (cho front-end).
- Java, C#, Python, PHP, Ruby, GO... (cho back-end).

2. Framework và thư viện và ngôn ngữ

- **Front-end:** React.js, Angular, Vue.js, Bootstrap.
- **Back-end:** Django hoặc Flask (Python), Spring Boot (Java), .NET Core (C#), Laravel (PHP)

3. Cơ sở dữ liệu

- **SQL:** MySQL, PostgreSQL, SQL Server.
- **NoSQL:** MongoDB, Firebase.

4. Công cụ hỗ trợ

- **Postman:** Test API (không test được lỗi CORS).
- **Swagger UI:** Test API trong trình duyệt, test lỗi CORS.
- **Git:** Quản lý phiên bản mã nguồn.
- **Ngrok:** chia sẻ Web apps chạy localhost

5. Dịch vụ lưu trữ Web

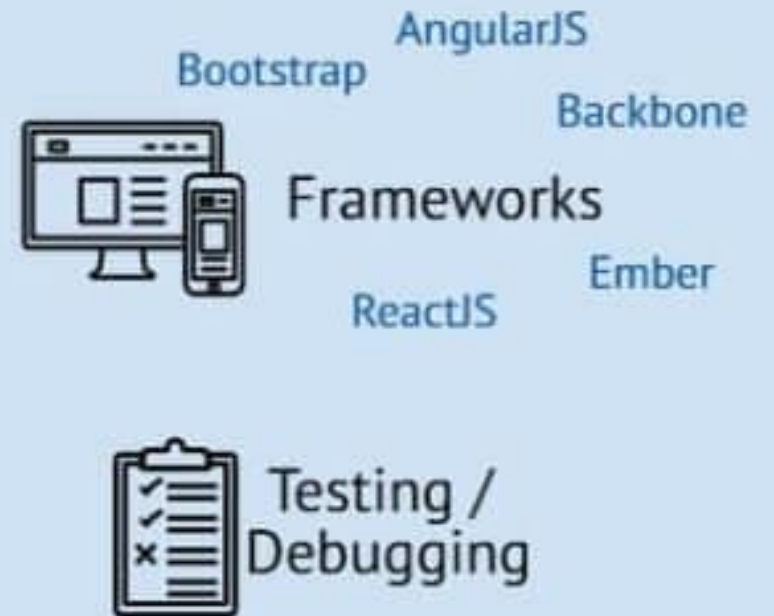
- AWS, Azure, Google Cloud, Heroku, Netlify.

FRONT END TOP SKILLS

HTML

CSS

JS



Front-End là phần giao diện của một ứng dụng Web hoặc phần mềm mà người dùng trực tiếp nhìn thấy và tương tác. Đây là lớp hiển thị và trải nghiệm người dùng (User Interface - UI), nơi các nội dung và chức năng được trình bày dưới dạng đồ họa, văn bản, hình ảnh, nút bấm, và các thành phần khác trên trình duyệt hoặc thiết bị.

- 1. HTML (HyperText Markup Language)**
- 2. CSS (Cascading Style Sheets)**
- 3. JavaScript (JS)**

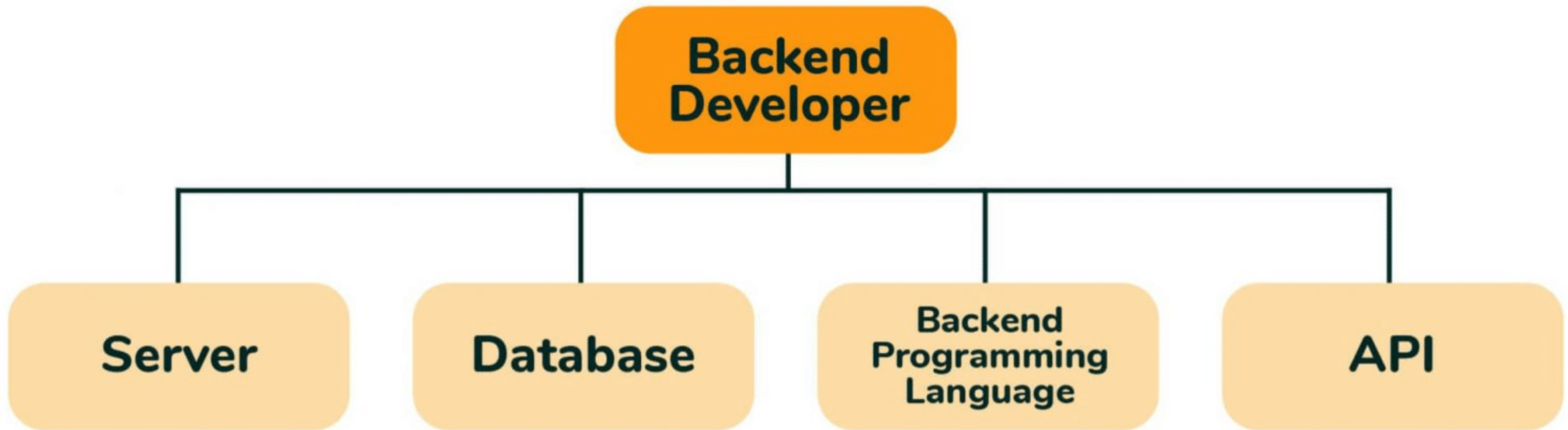
Front-End ngày nay không chỉ dừng lại ở HTML, CSS, và JavaScript mà còn được hỗ trợ bởi các công nghệ hiện đại.

- **React.js:** Thư viện xây dựng giao diện người dùng dựa trên thành phần (components).
- **Angular.js:** Framework mạnh mẽ phát triển ứng dụng Web đơn trang (SPA).
- **Vue.js:** Framework linh hoạt và dễ học để xây dựng giao diện tương tác.
- **Node.js**
- **Không sử dụng JS có thể kể đến Flutter (ngôn ngữ Dart)**

- **Bootstrap:** Bộ công cụ thiết kế sẵn với các thành phần giao diện và grid system.
- **Tailwind CSS:** CSS utility-first giúp tùy chỉnh giao diện linh hoạt

- **Webpack, Parcel:** Công cụ đóng gói (bundler) mã nguồn.
- **npm, Yarn:** Quản lý các thư viện và gói JavaScript

Web back-end là phần "hậu trường" của một ứng dụng Web, chịu trách nhiệm xử lý logic, quản lý dữ liệu, và giao tiếp với các hệ thống khác để cung cấp nội dung hoặc chức năng cho người dùng. Đây là phần nằm phía **server-side** (phía máy chủ), không hiển thị trực tiếp với người dùng, nhưng đóng vai trò thiết yếu trong hoạt động của toàn bộ ứng dụng



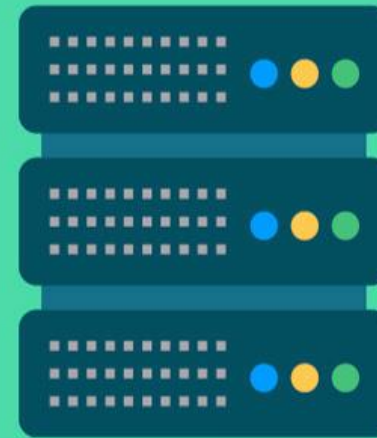
1. Máy chủ (Server)
2. Ứng dụng back-end
3. Cơ sở dữ liệu (Database)
4. API (Application Programming Interface) hoặc MVC (Model-View-Controller)

1. Xử lý yêu cầu từ người dùng
2. Giao tiếp với cơ sở dữ liệu
3. Quản lý logic nghiệp vụ
4. Bảo mật
5. Tích hợp với các dịch vụ bên ngoài

Front-end vs. Back-End



FRONT END



BACK END

Front-end vs. Back-end



Front End

- Markup and web languages such as HTML, CSS and Javascript
- Asynchronous requests and Ajax
- Specialized web editing software
- Image editing
- Accessibility
- Cross-browser issues
- Search engine optimisation



Back End

- Programming and scripting such as Python, Ruby and/or Perl
- Server architecture
- Database administration
- Scalability
- Security
- Data transformation
- Backup

Sự khác biệt giữa Back-End và Front-End

Tiêu chí	Front-End	Back-End
Chức năng chính	Hiển thị giao diện, tương tác với người dùng.	Xử lý logic, quản lý dữ liệu, bảo mật.
Công nghệ	HTML, CSS, JavaScript, React, Angular, Vue.	Java, C#, Python, Node.js, PHP.
Giao tiếp	Gửi yêu cầu đến server qua HTTP/S.	Nhận yêu cầu từ client, gửi phản hồi.
Môi trường chạy	Trình duyệt của người dùng	Server hoặc cloud
Kết nối	Gửi/nhận dữ liệu qua API	Cung cấp API và xử lý yêu cầu từ Front-End

1. Framework phổ biến
2. Cơ sở dữ liệu
3. Web Server
4. API và Giao thức

1. Framework phổ biến

- Web API .NET core, Web MVC .NET core (ASP.NET Core, C#)
- Spring Boot (Java)
- Node.js (JavaScript)
- Django/Flask (Python)
- Laravel (PHP)

2. Cơ sở dữ liệu:

- SQL: MySQL, PostgreSQL, SQL Server...
- NoSQL: MongoDB, Firebase (CSDL thời gian thực), Redis...

3. Web Server:

Nginx, Apache, Microsoft IIS.

4. API và Giao thức:

- RESTful API, SOAP
- HTTP/HTTPS, WebSocket

So sánh lập trình Web với lập trình ứng dụng desktop (Winform)

Tiêu chí	Lập trình Web	Lập trình Desktop (WinForms)
Môi trường chạy	Chạy trên trình duyệt (không cần cài đặt phần mềm trên máy khách).	Chạy trên máy tính cục bộ, cần cài đặt phần mềm trên hệ điều hành.
Ngôn ngữ và công cụ	HTML, CSS, JavaScript, các framework Web (ASP.NET, Spring Boot, Django, Node.js, v.v.).	Ngôn ngữ lập trình như C#, VB.NET (với WinForms hoặc WPF), Java (Swing, JavaFX).
Triển khai (Deployment)	Dễ dàng triển khai qua server Web, có thể cập nhật đồng thời cho tất cả người dùng.	Yêu cầu cài đặt trên từng máy tính, cập nhật cần phân phối bản mới hoặc qua các công cụ hỗ trợ (ClickOnce, MSI).

So sánh lập trình Web với lập trình ứng dụng desktop (Winform)

Tiêu chí	Lập trình Web	Lập trình Desktop (WinForms)
Phạm vi hoạt động	Có thể truy cập từ bất kỳ thiết bị nào có trình duyệt và kết nối internet.	Chỉ chạy trên máy đã cài đặt ứng dụng.
Tương tác giao diện	Thường giao diện đơn giản, tương tác qua trình duyệt, hỗ trợ responsive cho nhiều loại thiết bị.	Giao diện phong phú hơn, khả năng tùy biến cao nhờ kiểm soát chặt chẽ tài nguyên hệ thống.
Hiệu năng	Phụ thuộc vào tốc độ kết nối mạng và khả năng xử lý của server, cần tối ưu hóa tải dữ liệu.	Hiệu năng cao hơn do chạy cục bộ, không phụ thuộc vào mạng.

So sánh lập trình Web với lập trình ứng dụng desktop (Winform)

Tiêu chí	Lập trình Web	Lập trình Desktop (WinForms)
Khả năng tích hợp hệ thống	Hạn chế truy cập tài nguyên hệ thống (do bảo mật trình duyệt và sandboxing).	Dễ dàng truy cập và tích hợp với các tài nguyên hệ thống như registry, file system, thiết bị ngoại vi (scanner...).
Bảo mật	Dễ bị tấn công từ xa (SQL Injection, XSS, CSRF), cần cơ chế bảo mật phức tạp.	Bảo mật phụ thuộc vào môi trường cục bộ, ít bị tấn công từ xa hơn.

So sánh lập trình Web với lập trình ứng dụng desktop (Winform)

Tiêu chí	Lập trình Web	Lập trình Desktop (WinForms)
Khả năng mở rộng (Scalability)	Dễ mở rộng với nhiều người dùng bằng cách nâng cấp server hoặc sử dụng các dịch vụ đám mây.	Khó mở rộng do phụ thuộc vào cấu hình máy cục bộ và cần cài đặt từng máy.
Chi phí phát triển	Thường tốn kém hơn ban đầu, cần nhiều kỹ năng về cả back-end và front-end, nhưng dễ bảo trì và cập nhật.	Chi phí phát triển thấp hơn nếu dành cho nhóm người dùng nhỏ, nhưng bảo trì và cập nhật phức tạp.

So sánh lập trình Web với lập trình app mobile

Tiêu chí	Lập trình Web	Lập trình Ứng dụng Mobile
Môi trường chạy	Chạy trên trình duyệt (không cần cài đặt ứng dụng).	Chạy trên các nền tảng mobile như iOS, Android (yêu cầu cài đặt từ App Store hoặc Google Play).
Ngôn ngữ và công cụ	HTML, CSS, JavaScript, framework Web (React, Angular, Vue.js, ASP.NET, Spring Boot).	Native: Swift (iOS), Kotlin/Java (Android). Cross-platform: Flutter, React Native, Xamarin.
Khả năng truy cập	Truy cập từ bất kỳ thiết bị nào có trình duyệt và kết nối internet.	Chỉ chạy trên thiết bị đã cài đặt ứng dụng.

So sánh lập trình Web với lập trình app mobile

Tiêu chí	Lập trình Web	Lập trình Ứng dụng Mobile
Khả năng ngoại tuyến	Hạn chế, chỉ hoạt động offline qua caching (Service Workers, PWA).	Có khả năng ngoại tuyến tốt hơn nếu được lập trình phù hợp.
Triển khai (Deployment)	Dễ triển khai qua server, cập nhật đồng thời cho tất cả người dùng.	Yêu cầu phát hành qua các cửa hàng ứng dụng (App Store, Google Play), có thể chậm do quy trình kiểm duyệt.
Hiệu năng	Phụ thuộc vào tốc độ mạng và khả năng xử lý của server.	Hiệu năng tốt hơn do chạy native trên thiết bị, tận dụng tối đa tài nguyên phần cứng.

So sánh lập trình Web với lập trình app mobile

Tiêu chí	Lập trình Web	Lập trình Ứng dụng Mobile
Khả năng tương tác hệ thống	Hạn chế truy cập tài nguyên hệ thống (sandboxed trình duyệt).	Có khả năng truy cập tài nguyên hệ thống như GPS, camera, danh bạ, cảm biến, và phần cứng khác.
Bảo mật	Dễ bị tấn công từ xa (XSS, CSRF), cần triển khai cơ chế bảo mật phức tạp.	Ứng dụng native thường bảo mật tốt hơn do chạy trong sandbox riêng, nhưng vẫn có nguy cơ từ jailbreak hoặc root.
Chi phí phát triển	Thấp hơn, do chỉ cần phát triển một lần và chạy trên nhiều thiết bị qua trình duyệt.	Cao hơn, nhất là với ứng dụng native vì phải phát triển riêng cho từng nền tảng (iOS, Android).

So sánh lập trình Web với lập trình app mobile

Tiêu chí	Lập trình Web	Lập trình Ứng dụng Mobile
Khả năng mở rộng (Scalability)	Dễ mở rộng bằng cách nâng cấp server hoặc sử dụng dịch vụ đám mây.	Phụ thuộc vào khả năng xử lý của thiết bị người dùng, khó mở rộng đồng thời cho số lượng lớn người dùng.
Cập nhật	Dễ dàng cập nhật qua server mà không cần sự can thiệp của người dùng.	Yêu cầu người dùng cập nhật ứng dụng qua các cửa hàng ứng dụng.
Trải nghiệm người dùng (UX)	Thường hạn chế hơn, ít khả năng tùy biến giao diện và tương tác người dùng nâng cao	Trải nghiệm mượt mà, tối ưu hóa tốt hơn cho thiết bị di động.

Full-Stack Developer là lập trình viên có khả năng làm việc cả ở phần giao diện người dùng (**Front-End**) lẫn phần logic và cơ sở dữ liệu phía máy chủ (**Back-End**). Họ nắm vững các công nghệ, công cụ liên quan đến việc phát triển toàn bộ ứng dụng Web, từ thiết kế giao diện đến xử lý dữ liệu và triển khai ứng dụng.

1. Phát triển Front-End (Giao diện người dùng)

- Xây dựng giao diện người dùng bằng **HTML**, **CSS**, **JavaScript**.
- Sử dụng các framework như **React.js**, **Angular**, **Vue.js** để tạo trải nghiệm người dùng mượt mà và thân thiện.
- Đảm bảo trang Web **responsive**, tương thích trên nhiều thiết bị và trình duyệt khác nhau.

2. Phát triển Back-End (Logic và xử lý dữ liệu)

- Thiết kế và triển khai logic server-side sử dụng các ngôn ngữ như **Node.js, Java, Python, C#, PHP, Ruby hoặc Go**.
- Tạo và quản lý **API** để kết nối Front-End với Back-End.
- Xử lý xác thực người dùng, phân quyền, và các chức năng server-side khác.

3. Quản lý cơ sở dữ liệu

- Thiết kế, xây dựng, và tối ưu hóa cơ sở dữ liệu.
- Làm việc với cả **SQL databases** (MySQL, PostgreSQL, SQL Server) và **NoSQL databases** (MongoDB, Firebase).
- Đảm bảo cơ sở dữ liệu có khả năng mở rộng và bảo mật.

4. Tích hợp hệ thống

- Kết nối các dịch vụ hoặc API bên ngoài, như thanh toán trực tuyến, bản đồ, hoặc dịch vụ của bên thứ ba.
- Sử dụng các giao thức như **REST**, **SOAP**, hoặc **WebSocket**.

5. Kiểm thử và gỡ lỗi

- Kiểm tra và sửa lỗi cho cả Front-End và Back-End.
- Đảm bảo ứng dụng hoạt động ổn định và hiệu suất tốt.

6. Triển khai và bảo trì

- Đưa ứng dụng lên server hoặc cloud (AWS, Azure, Google Cloud).
- Thiết lập CI/CD (Continuous Integration/Continuous Deployment) để triển khai nhanh chóng và tự động.
- Theo dõi hiệu suất ứng dụng và cập nhật tính năng mới.

Full stack Cross platform app development



Flutter

Frontend UI
(Web, Mobile, Desktop)

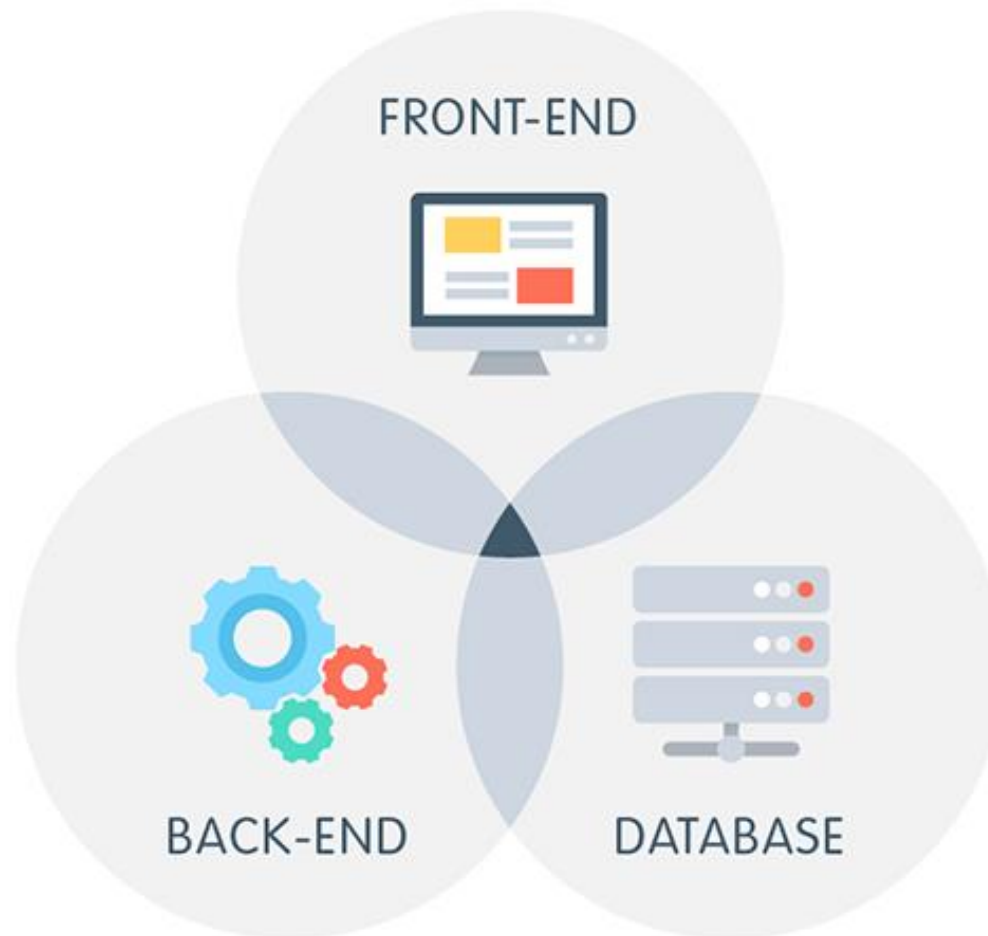


Backend
WebAPI



Database
SQL Express Edition

Lập trình full-stack



Lập trình Full-Stack là phương pháp phát triển toàn diện từ giao diện người dùng đến xử lý dữ liệu, bao gồm:

1. Xây dựng và quản lý Front-End

- ✓ Thiết kế giao diện bằng **HTML**, **CSS**, và **JavaScript**.
- ✓ Sử dụng các framework như **React.js**, **Angular**, **Vue.js** để tăng cường trải nghiệm người dùng (UX/UI).
- ✓ Đảm bảo giao diện **responsive** (phù hợp với mọi thiết bị).

2. Phát triển Back-End

- ✓Viết mã xử lý logic và tương tác với cơ sở dữ liệu bằng các ngôn ngữ server-side như **Node.js**, **Python**, **Java**, **PHP**, hoặc **C#**.
- ✓Xây dựng **API** để kết nối Front-End với Back-End.
- ✓Quản lý xác thực, phân quyền, và bảo mật.

3. Quản lý cơ sở dữ liệu

- ✓Thiết kế, triển khai, và tối ưu hóa cơ sở dữ liệu quan hệ (**MySQL**, **PostgreSQL**) hoặc phi quan hệ (**MongoDB**, **Redis**).
- ✓Đảm bảo tính toàn vẹn và bảo mật dữ liệu.

4. Triển khai và bảo trì

- ✓ Triển khai ứng dụng trên các nền tảng cloud như **AWS, Azure**, hoặc **Google Cloud**.
- ✓ Thiết lập và duy trì quy trình CI/CD (Continuous Integration/Continuous Deployment).
- ✓ Theo dõi, sửa lỗi, và tối ưu hóa hiệu năng hệ thống.

Sự khác biệt giữa Full-Stack Developer và Lập trình chuyên biệt

Tiêu chí	Full-Stack Developer	Lập trình chuyên biệt (Front-End/Back-End)
Phạm vi công việc	Làm việc trên toàn bộ ứng dụng: Front-End, Back-End, cơ sở dữ liệu, và triển khai.	Chỉ tập trung vào một mảng: Front-End hoặc Back-End.
Hiệu quả	Phù hợp cho dự án nhỏ hoặc nhóm ít người.	Tối ưu cho dự án lớn với đội ngũ chuyên môn hóa.
Mức độ chi tiết	Có thể thiếu chuyên môn sâu trong một số mảng nhất định.	Chuyên sâu, tối ưu hiệu suất trong lĩnh vực họ phụ trách.
Ứng dụng phổ biến	Startups, dự án nhỏ hoặc MVP (Minimum Viable Product).	Các dự án lớn, yêu cầu nhiều thành phần phức tạp.

- Interface trong lập trình là một bản thiết kế các hành vi (phương thức) mà các lớp phải triển khai.
- Tổ chức interface giúp tách biệt giao diện với thực thi, tăng tính linh hoạt và dễ bảo trì của hệ thống.

Bộ 5 nguyên lý thiết kế hướng đối tượng:

- **S**: Single Responsibility Principle (Nguyên lý trách nhiệm duy nhất).
- **O**: Open/Closed Principle (Mở để mở rộng, đóng để chỉnh sửa).
- **L**: Liskov Substitution Principle (Thay thế Liskov).
- **I**: Interface Segregation Principle (Tách biệt Interface).
- **D**: Dependency Inversion Principle (Đảo ngược phụ thuộc).

- Hệ thống quản lý gói (package) dành cho các dự án .NET.
- Cho phép tích hợp các thư viện, framework một cách dễ dàng vào ứng dụng

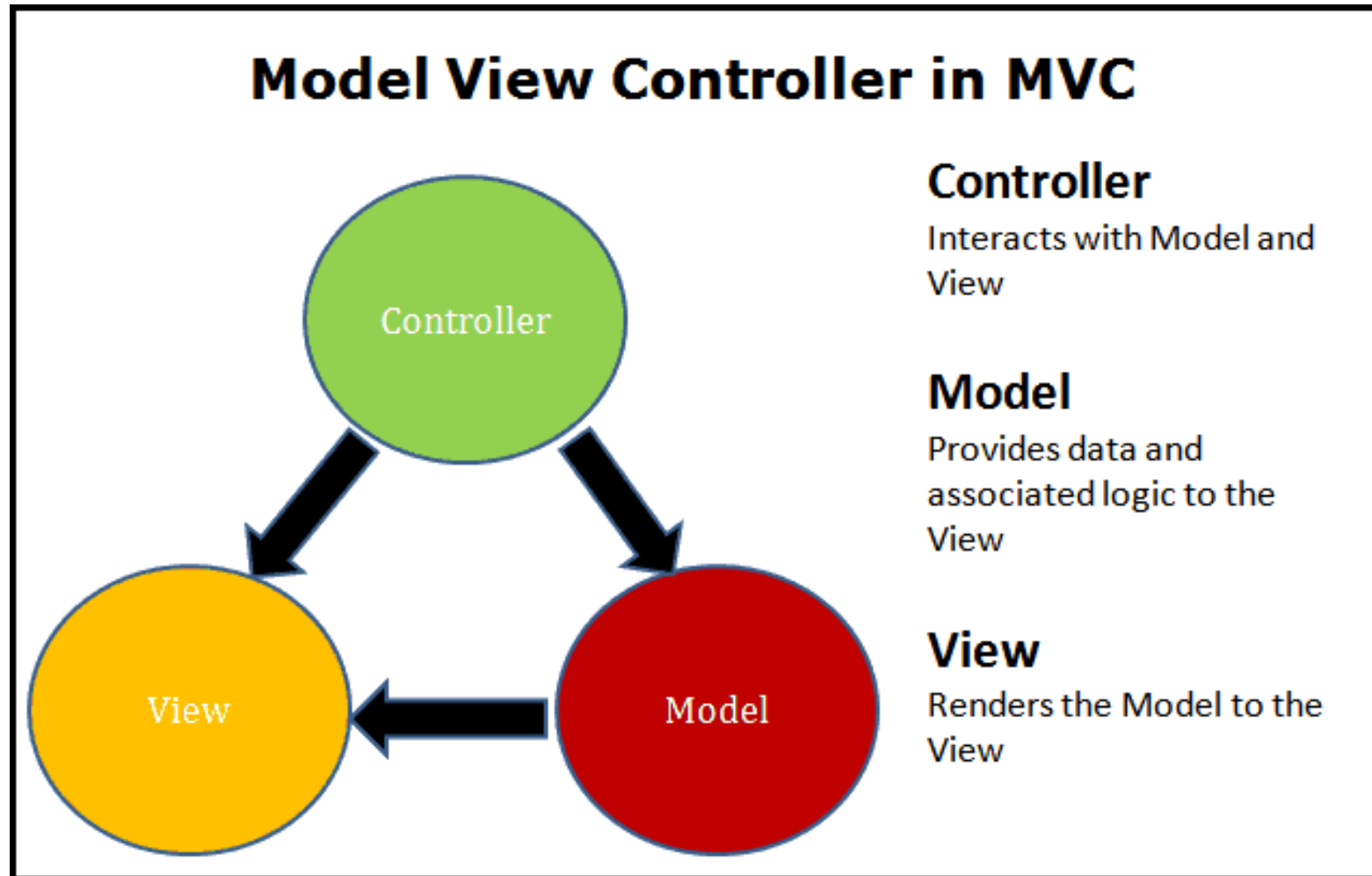
- JSON (JavaScript Object Notation): Định dạng trao đổi dữ liệu giữa client và server.
- Cấu trúc dạng key-value, dễ đọc và sử dụng rộng rãi

- Tạo giao diện người dùng với các form nhập liệu.
- Quản lý sự kiện như click, submit.
- Kiểm tra tính hợp lệ (validate) dữ liệu đầu vào

- ORM (Object-Relational Mapping) cho .NET, giúp ánh xạ giữa cơ sở dữ liệu và đối tượng lập trình.
- Hỗ trợ thực hiện CRUD mà không cần viết nhiều câu lệnh SQL
- Kiểm soát các câu lệnh lỗi có thể gây ra lỗ hổng SQL injection.

- Các mẫu thiết kế phần mềm cung cấp giải pháp phổ biến cho các vấn đề lặp lại.
- Ví dụ: Singleton, Factory, Observer, Decorator, Strategy, Command, DAO, REPOSITORY...

MVC MODEL



Model-View-Controller: Mô hình tổ chức code cho ứng dụng:

- **Model:** Xử lý dữ liệu.
- **View:** Hiển thị giao diện.
- **Controller:** Điều phối logic giữa Model và View

- **Message Processing Architecture:** Xử lý thông điệp giữa các thành phần của hệ thống.
- **Dispatcher:** Thành phần điều phối, phân phối các thông điệp đến đúng nơi để xử lý

- Kỹ thuật lập trình bất đồng bộ giúp xử lý nhiều tác vụ mà không làm ngưng ứng dụng.
- Async/Await: Cách viết code bất đồng bộ dễ đọc và quản lý

- Phong cách lập trình tập trung vào hàm và tránh thay đổi trạng thái.
- Sử dụng các biểu thức **lambda**, hàm mũi tên, và xử lý dữ liệu với **stream**.

- API của JavaScript để thực hiện các yêu cầu HTTP như GET, POST từ client đến server

Async/Await trong JavaScript

- Cách xử lý bất đồng bộ trong JS, giúp code gọn gàng, dễ đọc hơn so với callback

- Cấu trúc HTML cho việc nhập liệu từ người dùng.
- Thuộc tính action (URL gửi dữ liệu), method (GET/POST) xác định cách gửi dữ liệu
- Bắt validate với điều khiển HTML Form: Dùng các thuộc tính HTML như **required**, **pattern** để kiểm tra dữ liệu đầu vào
- **Hành động Submit:** Hành động gửi dữ liệu từ form HTML đến server.

Truyền tham số qua URL, localStorage, sessionStorage, Cookies

- **URL**: Gửi tham số trong đường dẫn (query string).
- **localStorage**: Lưu trữ dữ liệu lâu dài trong trình duyệt (không mất khi đóng trình duyệt).
- **sessionStorage**: Lưu dữ liệu tạm thời, mất khi đóng tab.
- **Cookies**: Lưu trữ dữ liệu nhỏ, gửi kèm mọi request đến server



CMC UNIVERSITY

Aspire to Inspire the Digital World

THANK YOU